Ders 2 Genişletilmiş Notu: Asimptotik Notasyonlar ve Özyinelemeli Olmayan ve Özyinelemeli Algoritmaların Analizi

Ders 2 Genişletilmiş Notu: Asimptotik Notasyonlar ve Özyinelemeli Olmayan ve Özyinelemeli Algoritmaların Analizi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Büyüme Mertebeleri ve Üç Notasyon
- 2. Big O: Üst Sınır
- 3. Big Omega: Alt Sınır
- 4. Big Theta: İki Yönlü Sınır
- 5. Ardışık Parçalardan Oluşan Algoritmalar
- 6. Büyüme Mertebelerini Limitle Karşılaştırma
- 7. Özyinelemeli Olmayan Algoritmaların Analizi
- 8. Özyinelemeli Algoritmaların Analizi
- 9. Derste Çözülen Alıştırmalar
- Örnek Sorular

1. Büyüme Mertebeleri ve Üç Notasyon

Geçen hafta dersin genel işleyişi konuşulmuş, algoritma analizinde en iyi, en kötü ve ortalama durum senaryolarından söz edilmişti. Bu hafta bu konular bilimsel bir zemine oturtuluyor: asimptotik notasyonların tanımları veriliyor ve algoritmalar üzerinden analiz yapılıyor.

Bir algoritmayı tasarlayıp analiz ettiğimizde, genelde bilinen büyüme sınıflarından birine ya da bunların birleşimine düştüğünü görürüz. Örneğin bir matrisi baştan sona tarayan bir algoritma n^2 karmaşıklığında olur. Bu sınıflar büyüme mertebesine (order of growth) karşılık gelir. Büyüme hızı çoğunlukla işlenen verinin miktarına, bazen başka bir parametreye bağlıdır. Ayrıntılı analiz yapılınca bu parametreye bağlı bir formül çıkar; bu formül, veri büyüdükçe algoritmanın ne kadar hızlı kötüleştiğini gösterir.

Slayttaki sıralama şöyledir (ekrandan, 00:20:00):

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \cdots < 2^n < 3^n < \cdots < n^n$$

Bir algoritmanın bütün bileşenlerini ayrıntılı görüp matematiksel bir açıklama yapmak her zaman kolay olmaz. Böyle durumlarda üç notasyondan yola çıkılır; bunlar algoritmanın bu sıralamada aşağı yukarı nerede olduğunu gösterir:

- **Big O:** fonksiyonun üst sınırını (upper bound) verir. “Fonksiyon şu fonksiyonun altında bir yerdedir” demeyi sağlar. Hoca bunun bir tür “kötü durum” bilgisi olduğunu, ama geçen hafta anlatılan kötü durum senaryosuyla aynı şey olmadığını belirtti.
- **Big Omega:** fonksiyonun alt sınırını belirler.
- **Big Theta:** hocanın ifadesiyle en kullanışlı olanıdır. Big Theta elde edilemediğinde diğer ikisi üzerinden hareket edilir.

Ders sonrası not: Hoca Big Theta için "average bound" dedi. Big Theta, ortalama durum (average case) anlamına gelmez; fonksiyonu alttan ve üstten aynı $g(n)$ ile sınırlayan sıkı sınırdır (tight bound). En kötü, en iyi ve ortalama durum ise girdinin türüne göre yapılan ayrı bir sınıflamadır; her biri için ayrıca O, Ω ya da Θ yazılabilir.

2. Big O: Üst Sınır

Elimizde gerçel sayılardan gerçel sayılara tanımlı iki fonksiyon olsun: f ve g. Aşağıdaki eşitsizliği sağlayan bir c ve n_0 bulunabiliyorsa $f(n) \in O(g(n))$ denir:

$$f(n) \leq c\, g(n) \quad \text{her } n \geq n_0 \text{ için}$$

Burada c pozitif bir sabit, n_0 negatif olmayan bir tam sayı olmalıdır. Bu durumda $O(g(n))$, $f(n)$'in üst sınırını oluşturur.

2.1 Örnek: $2n + 3$

Slayttaki örnek (ekrandan, 00:10:00) şu soruyu sorar: "$f(n) \leq ?$" eşitsizliğinin sağına ne yazılabilir?

- $7n$ yazılabilir: $n \geq 1$ için sağlanır.
- $2n+3n = 5n$ yazılabilir: $n \geq 1$ için sağlanır. Burada $c = 5$, $g(n) = n$ olur ve $f(n) \in O(n)$ elde edilir.
- $4n^2 + 3n^2 = 7n^2$ yazılabilir: $c = 7$, $g(n) = n^2$, $n \geq 1$. Bu durumda $f(n) \in O(n^2)$.

Sağ tarafı yazarken iki şeye dikkat edilir: sağda tek bir terim olmalı ve eşitsizlik her zaman sağlanmalıdır.

$O(n)$ ve $O(n^2)$ teknik olarak ikisi de doğrudur. $100n^3$ de yazılabilir, yine doğru olur. Bunu yazmak "fonksiyonum n^3 karmaşıklığının altında bir yerde" demektir. Ama fonksiyonun kendi sınırından uzaklaştıkça elde edilen bağıntı daha az kullanışlı olur. Bu yüzden en kullanışlı bilgi $f(n) \in O(n)$'dir. Aynı fonksiyon için $O(2^n)$ de "olur" denebilir.

Peki $f(n) \in O(\log n)$ denebilir mi? Hayır. $\log n$ sıralamada solda kalır. Big O, bulunulan noktanın solundaki daha küçük seçenekleri eler; teknik olarak "şunun üzerinde bir yerdedir" der.

Ders sonrası not: Slaytta fonksiyon "2n*3" diye yazılmış (ekrandan, 00:10:00), hoca da ilk anlatımda "2n çarpı 3" diye okudu. Big Theta kısmında aynı örneği "2n artı 3" diye andı (00:14:13). $2n \cdot 3 = 6n$ hiçbir $n \geq 1$ için $5n$'den küçük ya da eşit olmaz; slayttaki $\leq 5n$ ancak $2n + 3$ için doğrudur (python ile denendi). Örnek fonksiyon büyük olasılıkla $2n + 3$'tür. Bu notta $2n + 3$ kullanıldı.

3. Big Omega: Alt Sınır

Tanım benzerdir: pozitif bir c ve negatif olmayan bir n_0 için her $n \geq n_0$'da

$$f(n) \geq c\, g(n)$$

sağlanıyorsa $f(n) \in \Omega(g(n))$ denir.

Aynı örnek için ($2n + 3$) sağa gelecek bir $g(n)$ ve c aranır:

- $1 \cdot n$ yazılabilir: $c = 1$ için çalışır.
- $\log n$ yazılabilir: $n_0 = 1$ ve $c = 1$ için çalışır.

Buna göre $f(n) \in \Omega(n)$ ve $f(n) \in \Omega(\log n)$, ikisi de doğrudur. Ama $\Omega(n^2)$ olamaz; bunu sağlayacak bir bağıntı yazılamaz. Böylece bu kez fonksiyon için bir alt sınır belirlenmiş olur: $2n+3$ fonksiyonu sıralamada n'in ve sağında bir yerdedir. Teknik olarak sınırı aşağıya indirmek mümkündür, ama yine en yakın olanı seçmek daha isabetli yorum yapma şansı verir.

4. Big Theta: İki Yönlü Sınır

Big Theta için f'nin hem sağında hem solunda sınır koyan fonksiyonlar bulunur. Aynı $g(n)$ iki ayrı katsayıyla çarpılır:

$$c_1\, g(n) \le f(n) \le c_2\, g(n) \quad \text{her } n \ge n_0 \text{ için}$$

$2n+3$ için $c_1 = 1$ ve $c_2 = 5$ alınırsa $n \le 2n+3 \le 5n$ olur ve $f(n) \in \Theta(n)$ elde edilir. Hoca bunu "fonksiyonun tam olarak ait olduğu sınıfı, iki sınırın kesişiminde gösteriyor" diye özetledi.

4.1 Örnek: $\frac{1}{2}n(n-1) \in \Theta(n^2)$ ispatı

Slayt (ekrandan, 00:16:50) ispatı iki adımda kurar.

Üst sınır. Fonksiyon $\frac{1}{2}n^2 - \frac{n}{2}$'dir. $\frac{1}{2}n^2 - \frac{n}{2} \le c_2\, g(n)$ olacak bir g ve c_2 aranır. $g(n) = n^2$ ve $c_2 = \frac{1}{2}$ seçilirse

$$\frac{1}{2}n^2 - \frac{n}{2} \le \frac{1}{2}n^2, \quad n \ge 0$$

sağlanır.

Alt sınır. $\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \ge c_1\, g(n)$ olacak bir c_1 aranır. $g(n) = n^2$ olması gerektiğini görmek kolaydır; c_1 üzerinde biraz düşünmek gerekir. Slayttaki deneme tablosu:

c_1	Sonuç
1	tutmaz
2	tutmaz
1/2	tutmaz
1/4	tutar

Böylece $c_1 = \frac{1}{4}$, $c_2 = \frac{1}{2}$ bulunur.

n_0 seçimi. c_1 ve c_2 yazıldıktan sonra değerler yerine konup denenir. $n = 1$ için tutmaz ($\frac{1}{2}\cdot 1 \cdot 0 = 0 \ge \frac{1}{4}$ yanlış). $n = 2$ için tutar ($1 \ge 1$), $n = 3$ için de tutar ($3 \ge 2{,}25$). Bağıntı belirli bir noktadan sonra hep çalışır. Hoca en yakın olanı, yani $n_0 = 2$'yi seçti (python ile $n = 2$'den itibaren doğrulandı).

Hoca burada bir genel ilkeyi vurguladı: bundan sonraki analizlerde ilgilenilen durum, eleman sayısının (ya da algoritmanın bağlı olduğu parametrenin) sonsuza gittiği durumdur. Bir sıralama yapılıyorsa dizinin 100 ya da 200 elemanlı olması fark yaratmaz; eleman sayısı binlere, yüz binlere, milyonlara gittikçe algoritmanın kötüleşmesi fark yaratır.

4.2 Örnek: $4n^2 + 5n + 4$

Slayttaki eşitsizlikler (ekrandan, 00:19:20):

- $4n^2 + 5n + 4 \ge n^2$, yani $\Omega(n^2)$,
- $4n^2 + 5n + 4 \le 9n^2$, yani $O(n^2)$,
- $n^2 \le 4n^2 + 5n + 4 \le 9n^2$, yani $\Theta(n^2)$.

Hoca O için $9n^2$'nin yazılabileceğini, n'in başlangıç değerinin makul seçilmesi gerektiğini söyledi. Python ile denendiğinde $4n^2 + 5n + 4 \le 9n^2$ eşitsizliği $n = 1$'de tutmaz ($13 > 9$), $n \ge 2$ için tutar. $O(n^3)$ de yazılabilirdi, ama en yakını seçildi. İki sınırın arasında kalan bölge Big Theta'ya karşılık gelir.

4.3 Örnek: $n^2 \log n + n$

Slayt (ekrandan, 00:20:00):

$$n^2 \log n \leq n^2 \log n + n \leq 5\, n^2 \log n$$

Sol ve sağdaki $g(n)$'ler aynıdır ($n^2 \log n$), yalnız katsayılar farklıdır. Eşitsizlik sağlandığı için fonksiyon $O(n^2 \log n)$, $\Omega(n^2 \log n)$ ve $\Theta(n^2 \log n)$'dir. Bu fonksiyonun yeri sıralamada n^2 ile n^3 arasıdır; karesel ile kübik arası.

Soru: İki önceki slayttaki $5n^2 \log n$'deki 5 nasıl bulundu?

Cevap: Sağ taraf için $c_2\, g(n)$ biçiminde bir ifade yazılması gerekiyor ve bu ifade $n^2 \log n + n$'den büyük olmalı. 5 yerine 4 ya da 6 da yazılabilirdi; hoca 5'i seçti. 5'in özel bir önemi yok. Formül şunu söyler: eşitsizliği sağlayan herhangi bir (c, n_0) çifti varsa, o g fonksiyonu Big O, Big Omega ya da Big Theta için kullanılabilir. Yapılan iş, böyle bir çiftin mümkün olup olmadığını test etmektir. Soruyu soran öğrenci 2'nin de olabileceğini söyledi, hoca onayladı.

4.4 Sınırların grafikteki görüntüsü

Big Theta hesaplanabildiyse, fonksiyonun büyüme hızı grafikte iki eğri arasında kalır. Bir n_0 seçilir; o noktadan sonra fonksiyon iki eğrinin arasına oturur. Üstteki eğri $c_2\, g(n)$'dir. c_2 çok daha büyük seçilip üst eğri daha yukarıda çizilebilir; bu teknik olarak doğrudur ve "algoritmamın verimliliği bu sınırın altında bir yerde" demektir. Ama sınır gerçeğe ne kadar yakın belirlenirse çıkarım o kadar doğru olur.

> **Ders sonrası not:** Hoca grafiği anlatırken alttaki eğri için "Big Theta, yani alt sınırım" dedi (00:21:31). Alttaki eğri $c_1\, g(n)$'dir ve alt sınırı (Big Omega tipinde sınırı) verir. Big Theta, iki eğri arasındaki bölgenin kendisidir.

5. Ardışık Parçalardan Oluşan Algoritmalar

Bir algoritmada arka arkaya çalıştırılan iki ayrı parça varsa, genel analiz nasıl yapılır? Hoca sonucu baştan söyledi: genel karmaşıklık, parçalardan karmaşıklığı yüksek olanınkidir.

İki parça $t_1(n)$ ve $t_2(n)$ olsun, $t_1(n) \in O(g_1(n))$ ve $t_2(n) \in O(g_2(n))$. O zaman

$$t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

5.1 İspat

İspat için dört değişken tanımlanır: a_1, b_1, a_2, b_2. $a_1 \leq b_1$ ve $a_2 \leq b_2$ ise

$$a_1 + a_2 \leq 2 \max\{b_1, b_2\}$$

olur. Big O tanımından $n \geq n_1$ için $t_1(n) \leq c_1 g_1(n)$, $n \geq n_2$ için $t_2(n) \leq c_2 g_2(n)$'dir. $c_3 = \max\{c_1, c_2\}$ ve $n_0 = \max\{n_1, n_2\}$ alınır. İki eşitsizlik toplanır; sağ tarafta c_1 ve c_2 yerine c_3 yazmak eşitsizliği bozmaz, çünkü sağ taraf zaten büyük olanı arar:

$$t_1(n) + t_2(n) \leq c_1 g_1(n) + c_2 g_2(n) \leq c_3\, [g_1(n) + g_2(n)] \leq c_3 \cdot 2 \max\{g_1(n), g_2(n)\}$$

Bu, $t_1 + t_2 \in O(\max\{g_1, g_2\})$ demektir; sabit $2c_3$, eşik n_0'dır.

5.2 Örnek: Sırala, sonra tara

Bir dizide aynı elemandan olup olmadığını anlamak için önce dizi sıralanır, sonra sıralı dizi taranır. Bir parça $\frac{1}{2}n(n-1)$ karşılaştırma yapar ve $O(n^2)$'dir; diğer parça diziyi bir kez tarar, en kötü durumda $n-1$ karşılaştırma yapar ve $O(n)$'dir. Teoreme göre toplam algoritma $O(n^2)$ olur. Hoca slayttaki bir yazım hatasını anlatırken düzeltti ("burayı iki düşmüş, düzelteyim, n^2 olacak").

> **Ders sonrası not:** Hoca anlatırken parçaların rollerini karıştırır gibi oldu: $\frac{1}{2}n(n-1)$'lik algoritmayı "işi bitiren" diye, sıralamayı "tüm diziyi tarıyorsanız $n-1$" diye andı. Örnek büyük olasılıkla şöyledir: sıralama $\frac{1}{2}n(n-1)$ karşılaştırma ile $O(n^2)$, sıralı dizide komşu elemanları karşılaştıran tarama $n-1$ karşılaştırma ile $O(n)$. Sonuç ($O(n^2)$) değişmez.

Bellek verimliliği (space efficiency) için hoca şunu söyledi: yalnız sıralamak ile sıralayıp taramak arasında bellek açısından fark uygulamaya bağlıdır, ama normalde fark etmemelidir; iş n'lik bir alanla çözülebilir.

Soru: Burada yapılan şey, baskın (dominant) terimi almak diye düşünülebilir mi? Eşitsizliğin sağ tarafında baskın olan terimi almış oluyoruz.

Cevap: Algoritmanın parçaları büyüme fonksiyonları olarak ifade edilip her birine terim denirse, evet, öyle olmuş olur.

6. Büyüme Mertebelerini Limitle Karşılaştırma

Big O, Big Omega ve Big Theta bir fonksiyonun karmaşıklık özelliklerini özetler. Ama iki fonksiyonun karmaşıklığı karşılaştırılmak istendiğinde yanıltabilir. Bu durumda limit kullanmak daha etkilidir. $t(n)$ ve $g(n)$ bölünür ve $n \to \infty$ limitine bakılır (ekrandan, 00:30:00):

$$\lim_{n\to\infty} \frac{t(n)}{g(n)} = \begin{cases} 0 & t(n)\text{'nin büyüme mertebesi } g(n)\text{'ninkinden küçük} \\ c & \text{büyüme mertebeleri aynı} \\ \infty & t(n)\text{'nin büyüme mertebesi daha büyük} \end{cases}$$

Slayttaki oklara göre ilk iki durum $t(n) \in O(g(n))$'yi, son iki durum $t(n) \in \Omega(g(n))$'yi, ortadaki durum $t(n) \in \Theta(g(n))$'yi verir. Limit, notasyonlardan daha kullanışlıdır. Hoca limit ve toplam (summation) formüllerinin birkaç slayt ileride verildiğini, temel olanları bilmekte fayda olduğunu söyledi.

6.1 Örnek: $\frac{1}{2}n(n-1)$ ile n^2

$$\lim_{n\to\infty} \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2}\lim_{n\to\infty} \frac{n^2-n}{n^2} = \frac{1}{2}\lim_{n\to\infty}\left(1-\frac{1}{n}\right) = \frac{1}{2}$$

Katsayı dışarı alınır, n parantezine alınır. $n \to \infty$ iken $\frac{1}{n} \to 0$ olur, parantez 1'e gider ve sonuç $\frac{1}{2}$ çıkar. Sonuç pozitif bir sabit olduğu için iki fonksiyonun büyüme mertebesi aynıdır: $\frac{1}{2}n(n-1) \in \Theta(n^2)$.

Bölme ters yapılsaydı (n^2 bölü $\frac{1}{2}n(n-1)$) sonuç değişmezdi; limit yine bir sabit (2) olurdu ve aynı mertebe sonucu çıkardı. Hoca limit işlemlerini hatırlamak için slayttaki bir alıştırmayı önerdi; sonucun 2 çıkacağını söyledi.

6.2 Örnek: $n!$ ile 2^n

Karmaşıklık sınıfı yüksek algoritmalarda görülen bu iki fonksiyonu karşılaştırmak için Stirling formülü kullanılır (ekrandan, 00:33:10):

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n \quad (n \text{ büyükken})$$

n'in çok büyük değerleri zaten ilgilendiğimiz değerlerdir. Limit:

$$\lim_{n\to\infty}\frac{n!}{2^n} = \lim_{n\to\infty}\frac{\sqrt{2\pi n}\,(n/e)^n}{2^n} = \lim_{n\to\infty}\sqrt{2\pi n}\,\frac{n^n}{2^n e^n} = \lim_{n\to\infty}\sqrt{2\pi n}\left(\frac{n}{2e}\right)^n = \infty$$

$n!$ yerine Stirling karşılığı yazılır; $(n/e)^n$'deki e^n paydaya geçirilir; payda ve paydada n. kuvvetler olduğu için tek bir $(n/2e)^n$ ifadesinde toplanır. Bu ifade sonsuza gider: $n!$ çok daha hızlı büyür.

Slayttaki soru: Big Omega notasyonu, $n!$ ile 2^n'in aynı büyüme mertebesine sahip olması olasılığını dışlayabilir mi? Notasyonla $n! \in \Omega(2^n)$ yazılır; bu doğrudur. Ama Big Omega tanımı gereği "eşit ya da büyük" der ve eşit olma durumunu eleyemez. Limit sonsuz çıktığında ise hangisinin büyük olduğu net görülür.

Hoca ilk derste gösterdiği temel algoritma sınıfları ve büyüme mertebeleri slaytına da kısaca döndü: n^3'ten sonra algoritmalar hızla kötüleşir, bu yüzden çözümler mümkün olduğunca daha optimal hale getirilmeye çalışılır.

7. Özyinelemeli Olmayan Algoritmaların Analizi

Geçen derste anlatıldığı gibi, bir algoritmanın karmaşıklığı hakkında çıkarım yapmak için önce bir temel işlem (basic operation) belirlenir. Algoritmada döngü varsa bu genelde döngünün içinde en çok çalıştırılan satırdır.

7.1 MaxElement

Dizideki en büyük elemanı bulan algoritma (ekrandan, 00:37:20):

```
ALGORITHM MaxElement(A[0..n-1])
//Determines the value of the largest element in a given array
//Input: An array A[0..n-1] of real numbers
//Output: The value of the largest element in A
maxval <- A[0]
for i <- 1 to n-1 do
    if A[i] > maxval
        maxval <- A[i]
return maxval
```

Temel işlem döngü içindeki karşılaştırmadır. $C(n)$ bu karşılaştırmanın kaç kez yapıldığını göstersin. i, 1'den $n-1$'e gider ve her adımda bir karşılaştırma yapılır:

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

Hoca bunu en basit sıralı (sequential) algoritma analizi olarak sundu.

7.2 Genel plan

Daha karmaşık algoritmalarda dikkat edilecekler:

- Girdi boyutunu gösteren parametreye karar verilir: takip edilecek parametre ne olmalı?
- Temel işlem belirlenir.
- Temel işlemin çalışma sayısı yalnız girdi boyutuna mı bağlı, diye bakılır. En kötü, ortalama ve en iyi durum senaryoları ayrışıyorsa hangisi üzerinden gidileceğine karar verilir; gerekirse her biri ayrı analiz edilir.
- Temel işlemin kaç kez çalıştığı toplam (sum) formülleriyle yazılır ve formüllerle sonuca, en azından büyüme mertebesini gösterecek noktaya ulaşılır.

7.3 Toplam formülleri ve logaritma

Bu analizlerde logaritma ve toplam formülleri kullanılır. Hoca bunların temel olduğunu, kullandıkça hatırlanacağını söyledi. Vurguladıkları:

- Birinci formül ($\sum_{i=l}^{u} 1 = u - l + 1$) çok sık kullanılacak.
- Genelleştirilmiş formül de ($\sum_{i=1}^{n} i^k \approx \frac{1}{k+1} n^{k+1}$) sık kullanılacak.
- Sabit çarpanlar toplamın dışına çıkarılabilir. Alt ve üst sınırlara (i'nin nereden nereye gittiğine) dikkat etmek gerekir.
- Manipülasyon kuralları: bir toplam iki parçaya ayrılabilir; sınırlar değiştirilerek ifade bilinen formüllere uydurulabilir.
- Özellikle logaritmik karmaşıklıklı algoritmalarda aşağı ya da yukarı yuvarlama (floor, ceiling) gerekir; bunlarla ilgili formüller de slayttadır.

7.4 UniqueElements

Dizideki bütün elemanların birbirinden farklı olup olmadığını kontrol eden algoritma (ekrandan, 00:40:00):

```
ALGORITHM UniqueElements(A[0..n-1])
//Determines whether all the elements in a given array are distinct
//Input: An array A[0..n-1]
//Output: Returns "true" if all the elements in A are distinct
//        and "false" otherwise
for i <- 0 to n-2 do
    for j <- i+1 to n-1 do
        if A[i] = A[j] return false
return true
```

İki `for` döngüsü ve bir karşılaştırma vardır; temel işlem karşılaştırmadır. İki döngü gerekir, çünkü bir eleman diğer bütün elemanlarla karşılaştırılmalıdır. Daha verimli olsun diye ikinci döngüde j baştan başlatılmaz, i'nin kaldığı yerden devam eder.

Önce iç döngünün işlemleri tamamlanır: j, $i+1$'den $n-1$'e gider ve her adımda bir işlem yapılır. Bu, i sıfırdan $n-2$'ye giderken tekrarlanır. Formül en kötü durumu gösterir; en iyi durumla ilgilenilseydi ilk karşılaştırmada çıkılırdı ve sonuç 1 olurdu. Analiz en kötü durum üzerinden yürür.

Türetme (ekrandan, 00:43:20). Kullanılan formüller: $\sum_{i=l}^{u} 1 = u-l+1$ ve $\sum_{i=0}^{n} i = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i)$$

İç toplam için üst sınırdan alt sınır çıkarılıp 1 eklenir, $n-1-i$ bulunur. Sonra n'e bağlı ve i'ye bağlı kısımlar ayrılır:

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i = (n-1) \sum_{i=0}^{n-2} 1 - \frac{(n-2)(n-1)}{2}$$

$n-1$ sabit çarpan olarak dışarı çıkar; sol tarafta 0'dan $n-2$'ye giden toplamın içinde 1 kalır. Sağdaki toplam için $\frac{n(n+1)}{2}$ formülünde n yerine $n-2$ yazılır.

$$= (n-1)^2 - \frac{(n-2)(n-1)}{2} = \frac{(n-1)n}{2} \approx \frac{1}{2} n^2 \in \Theta(n^2)$$

İlk haftalarda anlatıldığı gibi, sabit toplama ve çıkarma terimleri ile sabit katsayılar elenebilir; asıl bağımlılık n'edir. Python ile $n = 2, \ldots, 9$ için karşılaştırma sayısının $\frac{n(n-1)}{2}$'ye eşit olduğu doğrulandı.

7.5 Matris çarpımı

İki matrisin tanımdan çarpımı (ekrandan, 00:45:20):

```
ALGORITHM MatrixMultiplication(A[0..n-1, 0..n-1], B[0..n-1, 0..n-1])
//Multiplies two square matrices of order n by the definition-based algorithm
//Input: Two n x n matrices A and B
//Output: Matrix C = AB
for i <- 0 to n-1 do
    for j <- 0 to n-1 do
        C[i, j] <- 0.0
        for k <- 0 to n-1 do
            C[i, j] <- C[i, j] + A[i, k] * B[k, j]
return C
```

İç içe üç `for` vardır. İlk iki döngü matrisleri tarar; en içteki döngü çarpılacak elemanları bulup iteratif bir toplama yapar. Bu toplamanın başlangıç değeri ikinci döngüden sonra verilir. Analizde önce en içteki döngüden başlanır: 0'dan $n-1$'e gider ve her seferinde bir çarpma yapılır.

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3$$

Her adımda bir n çarpanı dışarı çıkar; algoritma n^3 karmaşıklığındadır.

7.6 Binary: ikilik basamak sayısı

Aradan sonra özyinelemeli olmayan bir örnekle devam edildi. Pozitif bir n sayısının ikilik gösteriminde kaç basamak olduğunu hesaplayan algoritma (ekrandan, 01:04:40):

```
ALGORITHM Binary(n)
//Input: A positive decimal integer n
//Output: The number of binary digits in n's binary representation
count <- 1
while n > 1 do
    count <- count + 1
    n <- n/2
return count
```

Bu tür işlemlerde hep ikiye bölerek ilerlenir. Basamak sayısı 1 ile başlatılır; daha düşük olma şansı yoktur. n birden büyük olduğu sürece sayaç artırılır ve n ikiye bölünerek güncellenir. Bölme aşağı yuvarlanarak yapılır (tam sayı bölmesi). Algoritma ikinin kuvveti olan ve olmayan n değerleri için doğru çalışmalıdır.

İkinin kuvveti olmayan küçük bir değerle izleme, $n = 5$:

Adım	$n > 1$?	count	yeni n
Başlangıç		1	5
1	evet	2	$\lfloor 5/2 \rfloor = 2$
2	evet	3	$\lfloor 2/2 \rfloor = 1$
3	hayır	3	döngüden çıkılır

Sonuç 3'tür ($5 = 101_2$). Hoca burada tahta olmadan, yalnız slayt üzerinden anlatmanın zor olduğunu belirtti.

Döngü sayısı $\lfloor \log_2 n \rfloor$'dir. Formüldeki $+1$, döngüden çıkmadan önce yapılan son kontrolden gelir. Büyüme mertebesini gösteren fonksiyon $\lfloor \log_2 n \rfloor + 1$'dir; Big Theta karşılığı $\Theta(\log n)$'dir. Python ile $n = 1, 2, 3, 5, 8, 1000$ için algoritmanın sonucunun $\lfloor \log_2 n \rfloor + 1$'e eşit olduğu doğrulandı.

8. Özyinelemeli Algoritmaların Analizi

Özyinelemeli (recursive) algoritmalar analiz konuları içinde büyük yer tutar.

8.1 Faktöriyel

$n!$'i özyinelemeli hesaplayan basit bir algoritma: girdi olarak n alır. $n = 0$ olduğunda çağrıları bitirir ve 1 döndürür. Sıfıra ulaşmadıysa n'i bir azaltarak kendini tekrar çağırır; dönerken sonucu n ile çarparak döndürür. Fonksiyon şöyle ifade edilir:

$$F(n) = F(n-1) \cdot n$$

Analiz için recurrence relation yazılır. Recurrence relation, temel işlemin (bu örnekte çarpma) sayısını n'e bağlı olarak ifade eden bağıntıdır. Artık ilgilenilen şey sonuç değil, işlem sayısıdır; bu yüzden $M(n)$, $F(n)$'den farklıdır. $M(n)$, $F(n-1)$'i hesaplamak için gereken çarpımlara sondaki bir çarpımın eklenmesiyle bulunur:

$$M(n) = M(n-1) + 1 \quad (n > 0), \qquad M(0) = 0$$

Recurrence relation yazılırken başlangıç değeri, yani çağrıların nerede sonlanacağı iyi belirlenmelidir; analiz bunun üzerine kurulur. $n = 0$ olduğunda çarpma yapılmaz.

8.2 Backward substitution

Elde bir recurrence relation varken, sistematik bir yolla yalnız n'e bağlı (bir önceki n'e değil) kapalı bir formül bulmak gerekir. Bulunursa analiz oradan rahatça yapılır. Bunun farklı yöntemleri vardır: backward substitution, forward substitution. Derste backward substitution çözüldü. Fikir, algoritmanın özyinelemeli akışını anlamayı sağlayacak genel bir görüntü elde edilene kadar geriye doğru ilerlemektir.

Faktöriyel için (ekrandan, 01:10:00):

$$\begin{aligned} M(n) &= M(n-1) + 1 && \text{yerine koy: } M(n-1) = M(n-2) + 1 \\ &= [M(n-2) + 1] + 1 = M(n-2) + 2 && \text{yerine koy: } M(n-2) = M(n-3) + 1 \\ &= [M(n-3) + 1] + 2 = M(n-3) + 3 \end{aligned}$$

$M(n-1)$, $M(n-2)$ cinsinden yazılır; köşeli parantez içine konur, dışarıdaki 1 eklenir. Bir adım daha gidilince örüntü açıkça görülür. Genel biçim:

$$M(n) = M(n-i) + i$$

Başlangıç koşulu $n = 0$ için verildiğinden $i = n$ konur:

$$M(n) = M(n-1) + 1 = \cdots = M(n-i) + i = \cdots = M(n-n) + n = n$$

$M(0)$ işlem yapmadığı için sıfırdır. Faktöriyel fonksiyonunun karmaşıklığı $\Theta(n)$'dir.

8.3 Genel plan

Özyinelemeli bir fonksiyonun analizi için:

- Girdi boyutunu ifade eden parametreye odaklanılır.
- Temel işleme karar verilir.
- Temel işlemin farklı girdiler için farklı sayıda çalışıp çalışmadığına bakılır; en iyi, en kötü ya da ortalama durumdan hangisi seçilecekse onun üzerinden analiz edilir.
- Recurrence relation ve başlangıç koşulu (initial condition) kurulur.
- Büyüme mertebesine geçmek için recurrence çözülür.

8.4 Hanoi Kuleleri

Hoca bunu "güzel bir problem" diye tanıttı; şık çözümü özyinelemelidir. Webde animasyonla anlatan pek çok çözüm olduğunu, hatırlamak için bunlara bakılabileceğini söyledi.

Problem: n disk büyükten küçüğe sıralı biçimde bir çubuktadır; üç çubuk vardır. Diskler bir kaynaktan (source) bir hedefe (destination) aktarılacaktır. Aktarıldıktan sonra yine küçükler üstte, büyükler altta olmalıdır. Boşta kalan çubuk yardımcı olarak kullanılır (hangi çubuğun boşta olduğu değişir). Her hamlede küçük disk büyüğün üstünde olmalıdır; ters sıralama yapılmaz.

Çözümün mantığı n diski iki parçaya böler: tabandaki disk ve üstteki $n-1$ disk.

1. Üstteki $n-1$ disk yardımcı çubuğa yerleştirilir.
2. Tabandaki disk hedefe yerleştirilir.
3. Ayrılan $n-1$ disk hedefe yerleştirilir.
4. ve 3. adım tek hamlede yapılmaz; boşta kalan çubuk kullanılarak özyinelemeli yapılır.

Recurrence relation (ekrandan, 01:18:50): $n-1$ diskten oluşan parça iki kez aktarılır, tabandaki disk bir kez aktarılır.

$$M(n) = 2M(n-1) + 1 \quad (n > 1), \qquad M(1) = 1$$

$n = 1$ olduğunda artık çağrı yapılmaz; disk bir kez taşınır, bitiş koşulundaki 1 buradan gelir.

Backward substitution:

$$\begin{aligned} M(n) &= 2M(n-1) + 1 && \text{yerine koy: } M(n-1) = 2M(n-2) + \\ &= 2[2M(n-2) + 1] + 1 = 2^2 M(n-2) + 2 + 1 && \text{yerine koy: } M(n-2) = 2M(n-3) + \\ &= 2^2[2M(n-3) + 1] + 2 + 1 = 2^3 M(n-3) + 2^2 + 2 + 1 \end{aligned}$$

Dışarıdaki 2 köşeli parantezin içine de çarpıldığı için katsayılar 2^2, 2^3 diye büyür. Genel görüntü: 2^i ile çarpılan bir $M(n-i)$ ve 2^{i-1}'den 2^0'a kadar giden bir toplam:

$$M(n) = 2^i M(n-i) + 2^{i-1} + 2^{i-2} + \cdots + 2 + 1 = 2^i M(n-i) + 2^i - 1$$

Başlangıç koşulu $n = 1$'dir; bu $i = n - 1$ ile elde edilir:

$$M(n) = 2^{n-1} M(n-(n-1)) + 2^{n-1} - 1 = 2^{n-1} M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1$$

İki tane 2^{n-1}, 2^n eder. Büyüme mertebesi $\Theta(2^n)$'dir. Python ile $n = 1, \ldots, 11$ için recurrence'ın $2^n - 1$ verdiği doğrulandı.

8.5 Özyinelemeli çağrı ağacı

Özyinelemeli bir fonksiyonun nasıl çalıştığına dair içgörü elde etmenin bir yolu, çağrıları ağaç yapısında göstermektir. Her özyinelemeli çağrı bir düğüme karşılık gelir; düğüm, çağrıda kullanılan parametreyle adlandırılabilir.

Hanoi'de her n için iki çağrı yapılır ($2M(n-1)$). Kökte n vardır; iki $n-1$ çağrısı yapılır. Her $n-1$ de iki $n-2$ çağırır, böyle $n-3$'e doğru gider. $n = 1$ olduğunda çağrı yapılmaz; yapraklarda 1'ler vardır ve ağaç orada biter. Bu ağaçtaki düğüm sayısı karmaşıklığı verir. l seviyeyi göstermek üzere bir seviyedeki düğüm sayısı 2^l'dir: kökte (seviye 0) bir düğüm, seviye 1'de iki düğüm vardır ve böyle devam eder. Bütün düğümlerin toplamı çağrı sayısını verir.

> **Ek bilgi (derste anlatılmadı):** Ağacın seviyeleri 0'dan $n-1$'e gittiği için toplam düğüm sayısı $C(n) = \sum_{l=0}^{n-1} 2^l = 2^n - 1$'dir. Bu, backward substitution ile bulunan hamle sayısıyla aynıdır. Slayttaki formül derste "2 üzeri n eksi 1 üzerinden" diye anıldı; tam yazımı karede alınmadı.

8.6 BinRec: ikilik basamak sayısının özyinelemeli hali

Sıralı olarak bulunan ikilik basamak sayısı bu kez özyinelemeli yazılır (ekrandan, 01:20:00):

```
ALGORITHM BinRec(n)
//Input: A positive decimal integer n
//Output: The number of binary digits in n's binary representation
if n = 1 return 1
else return BinRec(floor(n/2)) + 1
```

n her seferinde ikiye bölünür ve basamak sayısını bulmak için dönüş değerine 1 eklenir. $n = 1$ olduğunda basamak sayısı bellidir; çağrı yapmadan çıkılır. Temel işlem toplamadır. Toplama sayısı için recurrence (ekrandan, 01:20:00):

$$A(n) = A(\lfloor n/2 \rfloor) + 1 \quad (n > 1), \qquad A(1) = 0$$

Dönüş değeri her seferinde 1 artırıldığı için bir toplama eklenir. $n = 1$ iken çağrı da toplama da yapılmaz, bu yüzden $A(1) = 0$'dır. Hoca bu tür analizlerde bunları doğru bulmanın önemli olduğunu söyledi. (Slaytın altındaki not alanında "3'ün binary hali 11" yazıyor.)

8.7 Smoothness rule

Aşağı yuvarlama olduğu için backward substitution burada kolay değildir. Recurrence içinde $n/2$ olması, ikinin kuvveti olmayan n değerlerinde çözümü zorlaştırır. Burada smoothness rule devreye girer. Kuralın formal bir tanımı vardır; hoca bunun üzerinden ilerleyen haftalarda geçebileceğini söyledi. Bu problem için temel söylediği şudur: $n = 2^k$ alınarak bulunan büyüme mertebesi, yukarı ya da aşağı yuvarlama işlemlerinden etkilenmez. Amaç genel bir bilgi elde etmek olduğundan, ikinin kuvveti olmayan bir n için hemen altına ya da üstüne yuvarlamak sonucu değiştirmez.

$n = 2^k$ konunca (ekrandan, 01:29:20) n'i ikiye bölmek k'yı bir azaltır:

$$A(2^k) = A(2^{k-1}) + 1 \quad (k > 0), \qquad A(2^0) = 0$$

Recurrence'tan çıkış koşulu $n = 1$ idi; $1 = 2^0$ olduğundan $k = 0$'da çıkılır. Backward substitution:

$$
\begin{aligned}
A(2^k) &= A(2^{k-1}) + 1 && \text{yerine koy: } A(2^{k-1}) = A(2^{k-2}) + 1 \\
&= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 && \text{yerine koy: } A(2^{k-2}) = A(2^{k-3}) + 1 \\
&= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3 \\
&\ \ \vdots \\
&= A(2^{k-i}) + i \\
&\ \ \vdots \\
&= A(2^{k-k}) + k
\end{aligned}
$$

Üsteki ifadeyi sıfıra indirmek için $i = k$ verilir:

$$A(2^k) = A(1) + k = k$$

$n = 2^k$ olduğundan $k = \log_2 n$, yani

$$A(n) = \log_2 n \in \Theta(\log n)$$

Hoca bu noktada konunun oturması için kendi örneklerini çözmenin ve formülleri kullanmanın gerektiğini vurguladı.

8.8 Fibonacci sayıları

Fibonacci sayıları çok bilinir: her sayı kendinden önceki iki sayının toplamıdır (ekrandan, 01:30:00):

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...$$

$$F(n) = F(n-1) + F(n-2) \quad (n > 1), \qquad F(0) = 0,\ F(1) = 1$$

```
ALGORITHM F(n)
//Computes the nth Fibonacci number recursively by using its definition
//Input: A nonnegative integer n
//Output: The nth Fibonacci number
if n <= 1 return n
else return F(n-1) + F(n-2)
```

$n \leq 1$ olduğunda n döndürülür (0 ya da 1). Her seferinde $n-1$ ve $n-2$ için çağrı yapılır. Hoca bunun optimum çözüm olmadığını, Fibonacci için optimizasyonu çok açık algoritmalar bulunduğunu, ama analizin nasıl yapıldığını görmek için güzel bir örnek olduğunu söyledi.

Backward substitution ile gitmek burada işleri karıştırır. Karşımıza homojen olmayan (inhomogeneous) recurrence çıkar. Bunları çözmek için çeşitli teknikler vardır; homojen olmayan recurrence'lar homojen olanlara çevrilerek yazılır. Hoca bununla ilgili ileride örnek çözebileceğini söyledi. Hocaya ilginç gelen nokta, bu işlemlerin sonunda altın oranın devreye girmesidir. Fibonacci dizisinde bir sayıyı kendinden önceki sayıya böldüğünüzde altın oranı elde edersiniz.

> **Ek bilgi (derste anlatılmadı):** Toplama sayısı için recurrence $A(n) = A(n-1) + A(n-2) + 1$, $A(0) = A(1) = 0$ biçimindedir; sondaki $+1$ yüzünden homojen değildir. $B(n) = A(n) + 1$ konunca homojen $B(n) = B(n-1) + B(n-2)$ elde edilir. Çözüm, $\phi = \frac{1+\sqrt{5}}{2} \approx 1{,}618$ olmak üzere ϕ^n ile büyür. Ardışık Fibonacci sayılarının oranı tam olarak ϕ değildir; n büyüdükçe ϕ'ye yaklaşır.

Çağrı ağacı. Backward substitution yerine ağaçla bakıldığında: $T(n)$, $T(n-1)$ ve $T(n-2)$'yi çağırır. $T(n-1)$, $T(n-2)$ ve $T(n-3)$'ü; $T(n-2)$ de $T(n-3)$ ve $T(n-4)$'ü çağırır. Çağrılar parametre 1 ya da 0 olduğunda biter. Bu yüzden her dalda sonlanma aynı yerde olmaz: bazı dallar en alta kadar inerken bazıları iki dallanmadan sonra biter. Altın oranın devreye girdiği yer budur.

Bu ağaç önümüzdeyken üst sınır nasıl hesaplanır? Üst sınır, fonksiyonun altında kalacağı bir fonksiyondur. Bütün düğümler dolu olsaydı, $n-1$ seviye olduğu için 2^n mertebesine ulaşılırdı. Ama burada bunun altında bir şey söz konusudur. Karmaşıklık, altın oran $\phi \approx 1{,}618$ ile $\Theta(\phi^n)$'dir. Bu, 2^n'den küçük, daha doğrusu gerçeğe daha yakın bir değerdir. 2^n Big O notasyonunda teknik olarak doğrudur.

> **Ders sonrası not:** Hoca önce "Big Omega'da 2^n evet, ama Big Theta'da 2^n değil" dedi (01:33:42). $\phi < 2$ olduğu için $\phi^n \notin \Omega(2^n)$'dir; doğru ifade $O(2^n)$'dir. Hoca birkaç cümle sonra "2^n Big O'da doğru" diyerek doğrusunu söyledi.

9. Derste Çözülen Alıştırmalar

Dersin son kısmında hoca alıştırmalar gösterdi. İlk ikisini (büyüme mertebesi karşılaştırmaları ve bir toplamın mertebesi) kendisi çözdü. Sonraki algoritma sorularında öğrencilere kâğıt kalemle düşünmeleri için birkaç dakika süre verdi, ardından fikirleri dinleyip çözümü paylaştı. Tahtaya çıkıp çözme imkânı olmadığı için en azından düşünme vakti olsun istedi. Algoritma sorularının hepsinde aynı beş soru sorulur:

- Bu algoritma ne hesaplar?
- Temel işlemi nedir?
- Temel işlem kaç kez çalışır?
- Algoritmanın verimlilik sınıfı nedir?
- Olası bir iyileştirme var mı?

Soruların metni ve çözümleri aşağıdaki "Örnek Sorular" bölümündedir. Hoca soru sormak isteyenlerin kendisinin sormasını beklemeden el kaldırmasını da rica etti.

Örnek Sorular

Soru 1 (ekrandan, 01:36:40): Aşağıdaki fonksiyon çiftlerinin büyüme mertebelerini karşılaştırın: (a) $n(n+1)$ ve $2000n^2$; (b) $100n^2$ ve $0{,}01n^3$; (c) $\log_2 n$ ve $\ln n$; (d) $(n-1)!$ ve $n!$.

> **Çözüm (derste verildi):** (a) $n(n+1) \approx n^2$ ve $2000n^2$: sabitler ilgilendirmez, büyüme hızları aynıdır. (b) $100n^2$ karesel, $0{,}01n^3$ kübiktir. Katsayının çok küçük olması önemli değildir; n'in küçük değerlerinde fark görülmeyebilir ama bir noktadan sonra n^3 daha yüksek mertebededir. (c) Taban değiştirme: $\log_b a = \frac{\log_x a}{\log_x b}$. e tabanına geçilirse $\log_2 n = \frac{\ln n}{\ln 2} = \frac{1}{\ln 2}\ln n$. $\log_2 n$, $\ln n$'nin sabit bir katıdır; mertebeleri aynıdır. (d) $n! = n\,(n-1)!$. Önceki şıklarda çarpanlar sabit olduğu için elenmişti; burada çarpan n'e bağlıdır. Bu yüzden $n!$ daha yüksek mertebededir.

Soru 2 (ekrandan, 01:38:50): Aşağıdaki toplamın büyüme mertebesini, $g(n)$ olabildiğince basit olacak biçimde $\Theta(g(n))$ ile bulun: $\sum_{i=0}^{n-1}(i^2+1)^2$.

> **Çözüm (derste verildi):** Kullanılan formül $\sum_{i=1}^{n} i^k \approx \frac{1}{k+1}n^{k+1}$. İçteki kare açılır ve toplam parçalara ayrılır. Mertebeyi en yüksek dereceli parça belirler:
>
> $$\sum_{i=0}^{n-1}(i^2+1)^2 = \sum_{i=0}^{n-1}(i^4+2i^2+1) \approx \sum_{i=0}^{n-1} i^4 = \sum_{i=1}^{n} i^4 + 0^4 - n^4$$
>
> Formül 1'den n'e gittiği için $i=0$ terimi eklenir, $i=n$ terimi çıkarılır:
>
> $$= \sum_{i=1}^{n} i^4 - n^4 \approx \frac{1}{4+1}n^{4+1} - n^4 = \frac{1}{5}n^5 - n^4 \approx \frac{1}{5}n^5 \in \Theta(n^5)$$
>
> Python ile toplamın $\frac{1}{5}n^5$'e oranı $n = 10, 100, 1000$ için 0,80; 0,98; 0,998 bulundu; oran 1'e yaklaşır.

Soru 3 (ekrandan, 01:40:00): Aşağıdaki algoritma için beş soruyu yanıtlayın.

```
ALGORITHM Mystery(n)
//Input: A nonnegative integer n
S <- 0
for i <- 1 to n do
    S <- S + i * i
return S
```

Çözüm (derste verildi, bir öğrenci cevapladı, hoca onayladı): Algoritma 1'den n'e kadar sayıların karelerinin toplamını hesaplar. Döngüye n kez girildiği için karmaşıklık $\Theta(n)$'dir. İyileştirme: bu toplamın matematiksel formülü bilinir; toplam tek seferde hesaplanabilir. Hoca formülü slaytta gösterdi ve işin $\Theta(1)$'de çözülebileceğini, algoritmanın ise $\Theta(n)$'de çözdüğünü söyledi.

Ek bilgi (derste anlatılmadı): Kareler toplamının kapalı formülü $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$'dır (örneğin $n = 10$ için 385; python ile döngü sonucuyla aynı çıktı). Formül sabit sayıda aritmetik işlemle hesaplandığı için $\Theta(1)$'dir.

Soru 4 (derste sözlü, 01:45:09; çözüm slaytı ekrandan, 01:47:00): Dizideki en büyük ve en küçük elemanı bulup farklarını döndüren algoritma için beş soruyu yanıtlayın. Hocanın ve bir öğrencinin anlatımına göre algoritma $i = 1$'den $n - 1$'e giden tek bir döngü içinde iki `if` ile en küçük ve en büyük değeri günceller, sonunda farklarını döndürür.

Çözüm (derste verildi): Bir öğrenci yanıtı verdi, hoca slayttaki çözümle doğruladı. Algoritma dizinin en büyük ve en küçük elemanlarının farkını döndürür. Temel işlem döngü içindeki karşılaştırmalardır; iki `if` aynı sayıda çalışır. Her biri $n-1$ kez yapılır:

$$C(n) = \sum_{i=1}^{n-1} 2 = 2(n-1) = 2n - 2 \approx 2n \in \Theta(n)$$

İyileştirme: Big Theta anlamında karmaşıklığı değiştirecek bir iyileştirme mümkün değildir. Ama küçük bir iyileştirme yapılabilir: $A[i] < minval$ kontrolünden sonraki `if` yerine `else if` konursa bazı durumlarda ikinci karşılaştırmaya girilmez. Bu verimliliğe az katkı sağlar, sonucu değiştirmez.

Soru 5 (ekrandan, 01:50:00): Aşağıdaki algoritma için beş soruyu yanıtlayın.

```
ALGORITHM Enigma(A[0..n-1, 0..n-1])
//Input: A matrix A[0..n-1, 0..n-1] of real numbers
for i <- 0 to n-2 do
    for j <- i+1 to n-1 do
        if A[i, j] = A[j, i]
            return false
return true
```

Çözüm (derste verildi): Algoritma bir matrisin simetrik olup olmadığını kontrol eder. Son eleman $n-1$. indisli olduğundan i 0'dan $n-2$'ye gider; içteki döngü $i+1$'den $n-1$'e gider ve $A[i,j]$ ile $A[j,i]$'yi karşılaştırır. Temel işlem bu karşılaştırmadır. İç döngüden 1'den başlamayan bir toplam, dış döngüden 0'dan $n-2$'ye giden bir toplam gelir. İç toplamda üst sınırdan alt sınır çıkarılıp 1 eklenir:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n-1-i) = (n-1)+(n-2)+\cdots+1 = \frac{(n-1)n}{2} \in \Theta(n^2)$$

İyileştirme: bu zaten verimli bir algoritmadır. Verimsiz bir yol, matrisin transpozunu alıp kendisiyle karşılaştırmak olurdu. Hoca o yolun da $\Theta(n^2)$ olacağını, ama aradaki farkın yarı yarıya olduğunu söyledi.

Ders sonrası not: Slayttaki koşul `if A[i, j] = A[j, i] return false` biçiminde yazılı. Hocanın anlattığı simetri kontrolü için koşul büyük olasılıkla $A[i,j] \neq A[j,i]$ olmalıdır; eşitlikle yazılırsa simetrik bir matriste ilk karşılaştırmada `false` döner.

Soru 6 (ekrandan, 02:00:00): Bir ızgarada 0. iterasyonda ortada 1×1'lik bir kare boyalıdır. Her iterasyonda boyalı alanın dışına, dört komşuluğa göre 1×1'lik kareler eklenir. n. iterasyonda (örneğin bininci iterasyonda) kaç kare boyalıdır, ve zaman karmaşıklığı nedir?

> **Çözüm (derste verildi):** Hoca öğrencilere kâğıt kalemle çizmeleri için süre verdi, sonra kareleri sayarak örüntüyü gösterdi. 1. iterasyonda 4 kare, 2. iterasyonda 8 kare, 3. iterasyonda 12 kare eklenir: dört komşuluk nedeniyle iterasyon numarasının 4 katı kadar. Slayttaki değerler: - $C(0) = 1$ - $C(1) = 5 = 1+4 = 1+4\cdot1 = C(0)+4\cdot1$ - $C(2) = 13 = 5+8 = 5+4\cdot2 = C(1)+4\cdot2$ - $C(3) = 25 = 13+12 = 13+4\cdot3 = C(2)+4\cdot3$
>
> Recurrence: $C(n) = C(n-1) + 4n$, $C(0) = 1$. Backward substitution:
>
> $$C(n) = C(n-1)+4n = C(n-2)+4(n-1)+4n = C(n-3)+4(n-2)+4(n-1)+4n$$
> $$= C(n-i)+4(n-i+1)+4(n-i+2)+\cdots+4n$$
>
> Başlangıç koşulu için $n-i=0$, yani $i=n$ konur ve $C(0)=1$ yazılır; 4 parantezine alınınca tanıdık bir toplam çıkar:
>
> $$C(n) = 1+4\sum_{i=1}^{n} i = 1+4\cdot\frac{n(n+1)}{2} = 2n^2+2n+1 \in \Theta(n^2)$$
>
> Python ile recurrence ve kapalı formül $n = 0, \ldots, 7$ için karşılaştırıldı: 1, 5, 13, 25, 41, 61, 85, 113; ikisi aynı çıktı.
>
> **Ders sonrası not:** Slaytta recurrence "for all $n \geq 0$" diye yazılı. $C(n-1)$ terimi $n = 0$ için tanımsız olduğundan bağıntı büyük olasılıkla $n \geq 1$ için, $C(0) = 1$ başlangıç koşuluyla geçerlidir.

Dersin Toparlaması

Bu derste geçen haftaki en iyi, en kötü ve ortalama durum kavramları biçimsel bir zemine oturtuldu. Big O (üst sınır), Big Omega (alt sınır) ve Big Theta (iki yönlü sıkı sınır) tanımları c ve n_0 sabitleriyle verildi; örneklerde en yakın sınırın en kullanışlı olduğu gösterildi. Ardışık parçalardan oluşan bir algoritmanın karmaşıklığını büyük olan parçanın belirlediği ispatlandı. İki fonksiyonu karşılaştırmak için limit yönteminin notasyonlardan daha bilgilendirici olduğu, $n!$ ile 2^n örneğinde Stirling formülüyle görüldü.

Dersin ikinci yarısında analiz yöntemleri uygulandı. Özyinelemeli olmayan algoritmalarda (MaxElement, UniqueElements, matris çarpımı, Binary) temel işlem belirlenip toplam formülleriyle sayıldı. Özyinelemeli algoritmalarda (faktöriyel, Hanoi Kuleleri, BinRec, Fibonacci) recurrence relation ve başlangıç koşulu kuruldu, backward substitution ile kapalı form bulundu. $n/2$'li recurrence'larda smoothness rule kullanıldı, Fibonacci'de altın oranın ortaya çıktığı görüldü. Ders, öğrencilerin üzerinde düşündüğü alıştırmalarla kapandı. Hoca bir sonraki hafta konulara devam edileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.